

INF1015 - Programmation orientée objet avancée

[Tableau de bord](#) / [Mes cours](#) / [INF1015 - Programmation orientée objet avancée](#) / Matière du cours / [Exercices sur les pointeurs](#)

Exercices sur les pointeurs

Exercices qui permettent de pratiquer:

- `new Type` // un emplacement utilisant le "default-initialization" qui laisse non initialisé si le type n'a pas de constructeur (dont les types de base).
- `new Type(valeur)` ou `new Type{valeur}` // un emplacement initialisé à valeur.
- `new Type[nombreElements]` // un emplacement pour un tableau, avec "default-initialization" (voir-ci-dessus) des éléments.
- `new Type[nombreElements](valeur1, valeur2, ...)` ou `new Type[nombreElements]{valeur1, valeur2, ...}` // un emplacement pour un tableau initialisé aux valeurs, s'il y a moins de valeurs données que `nombreElements`, les autres utilisent le "value-initialization" qui met à zéro pour les types de base, ou utilise le constructeur par défaut.
- `delete x;` // relâche l'emplacement pour un élément (alloué avec `new` sans `[]`).
- `delete[] x;` // relâche l'emplacement pour un tableau (alloué avec `new` avec `[]`).
- `*` -> `[]` // différentes manières de déréférencer un pointeur selon qu'il pointe vers un élément, une struct où on veut accéder à un membre, ou un tableau.
- `&` // l'adresse de quelque chose.
- Ne pas retrouver l'adresse de quelque chose qui n'existera plus au retour.

[Le solutionnaire est ici.](#)

L'exercice 2 a une version utilisant *unique_ptr* (vu au chap.6).

Exercice 1: Manipulation de pointeurs simples

Exercice pour se pratiquer avec `*`, `&` et `new`, et comprendre qu'une copie de pointeur ne copie pas ce qui est pointé.

Écrire un programme qui: (chaque étape prend généralement une ligne de programme)

- Alloue dynamiquement un `int` et conserve le pointeur '`x`'.
- Met la valeur 3 dans le `int` pointé par '`x`'.
- Affiche le `int` pointé par '`x`'. (Affiche 3)
- Copie le pointeur dans un autre pointeur '`y`'.
- Affiche le `int` pointé par '`y`'. (Affiche 3) Pourquoi?
- Met la valeur 4 dans le `int` pointé par '`y`'.
- Affiche encore le `int` pointé par '`x`'. (Affiche 4) Pourquoi?
- Copie le `int` pointé par '`x`' dans une nouvelle mémoire allouée dynamiquement et conserve le pointeur '`z`'.
- Met la valeur 5 dans le `int` pointé par '`x`'.
- Affiche les `int` pointés par '`x`' et '`z`'. (Affiche 5 et 4) Pourquoi?
- Passe le pointeur '`x`' à la fonction '`f`' (voir plus bas la description des fonctions).
- Affiche le `int` pointé par '`x`' et '`y`' (après que la fonction '`f`' ait terminé). (Affiche 6 et 6) Pourquoi?
- Définit une variable '`a`' de type `int` qui vaut 8.
- Passe la variable '`a`' à la fonction '`f`'.
- Affiche le contenu de '`a`'. (Affiche 6)
- Le main passe le pointeur '`x`' à la fonction '`g`'.
- Affiche le `int` pointé par '`x`' et '`y`'. (Affiche 10 et 9) Pourquoi?
- Tente de passer la variable '`a`' à la fonction '`g`'. Pourquoi il y a un problème?
- Fait une variable qui pointe vers '`a`' pour la passer à la fonction '`g`'. (prend 2 lignes)
- Affiche le contenu de '`a`' et le `int` pointé par la variable qui pointait vers '`a`'. (Affiche 9 et 10)
- (on ne demande pas de faire la désallocation)

- La fonction 'f' prend un pointeur de int comme seul paramètre et ne retourne rien:
 - Met 6 dans le int pointé par le paramètre.
 - Alloue dynamiquement un nouveau int et conserve le pointeur dans le paramètre, et met 7 dans ce int.
- La fonction 'g' est une copie de la fonction 'f' mais prend une référence à un pointeur de int comme seul paramètre (au lieu de directement un pointeur) et ne retourne rien. Et on change les valeurs 6 et 7 dans la fonction par 9 et 10.

Exercice 2: "Tableaux" de différentes sortes

Écrire différentes versions d'un programme où:

- Une fonction demande des valeurs réelles à l'utilisateur en lui demandant premièrement combien de valeurs il veut entrer.
- Une fonction pour afficher les valeurs lues.
- Un "main" qui utilise les fonctions pour lire puis afficher les valeurs.
- Il ne doit y avoir aucune fuite de mémoire (utiliser la bibliothèque du cours pour la vérification d'allocation, pour s'en assurer).

Si vous voulez uniquement pratiquer les pointeurs, faites la version 3 décrite ci-dessous. La fonction pour afficher les valeurs lues peut être la même pour toutes les versions si elle prend un *span* en paramètre au lieu du type spécifique à la version.

Version 1 (chap.1-2): Il y a un nombre maximum de valeurs, qui est déterminé lors de la compilation. Un tableau de taille fixe est donc utilisé et passé par adresse aux fonctions.

Les autres versions n'ont pas de nombre maximum de valeurs:

Version 2 (chap.1-2): On utilise `std::vector` pour s'occuper de conserver les valeurs.

Version 3 (chap.3): La mémoire est allouée dynamiquement à la main et retournée (en valeur de retour) par la fonction qui demande les valeurs.

Version 4 (chap.6): On utilise un `unique_ptr` pour gérer automatiquement la possession de la mémoire allouée par la fonction qui demande les valeurs.

(La meilleure version de ce programme est généralement la version avec `std::vector`, qui est lui-même implémenté en utilisant l'allocation dynamique)

Exercice 3: Différents double pointeurs

Exercice qui permet de pratiquer les *new* et *delete* avec et sans [], et faire les bonnes déréférences (* et []) aux bons endroits) pour différents double pointeurs.

- Allouer un pointeur vers un pointeur vers un int. Mettre 3 dans ce int et afficher ce int.
- Allouer un pointeur vers un tableau de 2 pointeurs, chacun pointant vers un int distinct. Mettre 4 et 5 dans ces ints et les afficher.
- Allouer un pointeur vers un pointeur vers un tableau de 2 ints. Mettre 6 et 7 dans ces ints et les afficher.
- Allouer un pointeur vers un tableau de 2 pointeurs qui pointent chacun vers un tableau distinct de 2 ints. Mettre 8,9 et 10,11 dans ces ints et les afficher.
- Désallouer tout pour qu'il n'y ait pas de fuite de mémoire (utiliser la bibliothèque du cours pour la vérification d'allocation, pour s'en assurer).

Exercice 4: Manipulation de pointeurs avec struct

Un peu similaire à l'exercice 1 mais avec une struct. Il y a au moins un endroit où l'opérateur `->` devrait être utilisé (ne pas faire `(*pointeur).champ`).

Soient les struct:

```
struct A {
    int x, y;
};
struct B {
    A* valeurs;
    int nValeurs;
};
```

(chaque étape prend généralement une ligne de programme)

- La fonction 'afficherA' prend un A en paramètre sans en faire de copie ni utiliser de pointeur:
 - Affiche ses valeurs de 'x' et 'y'.
- La fonction 'incrémenter' prend un A en paramètre sans en faire de copie ni utiliser de pointeur, pour pouvoir modifier la valeur de l'argument passé:
 - Incrémente les valeurs de 'x' et 'y' (2 opérations).
- La fonction 'afficherB' prend un B en paramètre sans en faire de copie:
 - Affiche les 'nValeurs' premières valeurs du tableau pointé par 'valeurs'.
- Le main:
 - Définit une variable 'a' de type A avec les valeurs 3,4.
 - Affiche 'a'. (Affiche 3,4)
 - Fait pointer la variable 'b' vers la variable 'a'.
 - Affiche le A pointé par 'b'. (Affiche 3,4) Pourquoi?
 - Affecte à ce qui est pointé par 'b' les valeurs 5,6 (en une opération).
 - Affiche 'a'. (Affiche 5,6) Pourquoi?
 - Incrémente (avec la fonction) ce qui est pointé par 'b'.
 - Affiche 'a'. (Affiche 6,7) Pourquoi?
 - Affiche juste la valeur du 'x' pointé par 'b'. (Affiche 6)
 - Alloue dynamiquement un A en y copiant ce qui est pointé par 'b' (d'un coup, pas en spécifiant le 'x' et 'y') et conserve dans le pointeur dans 'c'.
 - Incrémente (avec la fonction) ce qui est pointé par 'b'.
 - Affiche ce qui est pointé par 'b' et 'c' (2 opérations). (Affiche 7,8 et 6,7) Pourquoi?
 - Définit une variable 'd' de type B en allouant un tableau de 2 cases dont les valeurs sont celles pointées par b et c, et avec nValeurs qui vaut 2.
 - Affiche 'd'. (Affiche 7,8 et 6,7)
 - Copie 'd' d'un coup dans une variable 'e' (sans copier les valeurs du tableau).
 - Met 9 dans le 'x' de la deuxième valeur dans le tableau de 'e'.
 - Affiche 'd'. (Affiche 7,8 et 9,7) Pourquoi?
 - (on ne demande pas de faire la désallocation)

Exercice 5: Retour d'un pointeur à un «objet»

Compléter le TODO dans le programme:

```
#include <iostream>
#include <string>
using namespace std;

struct Personne { string prenom, nom; };
void afficherPersonne(const Personne& p) { cout << p.prenom << " " << p.nom; }

Personne* obtenirPersonne()
{
    Personne personne = { "Jane", "Doe" };

    //TODO: Retourner un pointeur de Personne pour Jane Doe (au lieu de nullptr), pour que le programme fonctionne correctement.
    return nullptr;
}

int main()
{
    Personne* p = obtenirPersonne();
    afficherPersonne(*p);
    delete p;
}
```

Modifié le: vendredi 6 mai 2022, 12:13